Lecture 26.06: Linear Regression GD (Step 4 — Update Rule & Full OOP Class)

📌 Topic Overview

In this lecture, Shraddha ma'am completes the Gradient Descent algorithm with **Step 4: Parameter Update Rules**, assembling all 4 steps into a clean, reusable Object-Oriented `LinearRegressionScratch` class.

🔄 The Parameter Update Rule

In every iteration (epoch), the parameters are updated along the negative gradient:

$$w := w - \alpha \frac{\partial J}{\partial w} = w - \alpha \cdot dw$$

$$b := b - \alpha \frac{\partial J}{\partial b} = b - \alpha \cdot db$$

Where α is the learning rate.

📊 Visual Flowchart: The Complete Gradient Descent Cycle



💻 Python Implementation

```
import numpy as np

class LinearRegressionScratch:
    """
    Complete Scratch Implementation of Linear Regression with Gradient Descent:
    Step 1: Initialize weights (w) and bias (b) to zeros.
    Step 2: Predict y_hat and calculate MSE cost.
    Step 3: Compute gradients (dw, db).
    Step 4: Update parameters: w = w - lr * dw, b = b - lr * db.
    """
    def __init__(self, learning_rate=0.01, epochs=1000):
        self.lr = learning_rate
        self.epochs = epochs
        self.w = None
        self.b = None
        self.cost_history = []

    def fit(self, X, y):
        m, d = X.shape
        # Step 1: Initialize
        self.w = np.zeros(d)
        self.b = 0.0
        self.cost_history = []

        for _ in range(self.epochs):
            # Step 2: Hypothesis & Cost
            y_hat = np.dot(X, self.w) + self.b
            cost = (1.0 / (2.0 * m)) * np.sum((y_hat - y) ** 2)
            self.cost_history.append(cost)

            # Step 3: Gradients
            error = y_hat - y
            dw = (1.0 / m) * np.dot(X.T, error)
            db = (1.0 / m) * np.sum(error)

            # Step 4: Parameter Updates
            self.w -= self.lr * dw
            self.b -= self.lr * db

        return self

    def predict(self, X):
        return np.dot(X, self.w) + self.b

# --- Function Call & Testing ---
np.random.seed(42)
X_synthetic = 2 * np.random.rand(100, 1)
# True relationship: y = 4 + 3x
y_synthetic = 4 + 3 * X_synthetic.squeeze() + np.random.randn(100) * 0.2

model = LinearRegressionScratch(learning_rate=0.1, epochs=500)
```

```
model.fit(X_synthetic, y_synthetic)

print(f"True slope: 3.0 | Learned Weight (w): {model.w[0]:.4f}")
print(f"True bias:  4.0 | Learned Bias (b):   {model.b:.4f}")
print(f"Initial Cost: {model.cost_history[0]:.4f} -> Final Cost: {model.cost_histc
```

🔑 Key Takeaways

1. Gradient descent iteratively refines weights until the loss plateaus.
2. An appropriate learning rate is vital: too high causes divergence, too low converges painfully slowly.
3. Our scratch implementation mimics Scikit-Learn's `fit()` and `predict()` API contract perfectly.